

HIMSHAKTI / KAVACH — SIH 2026 Project Plan

PS 26061 · AI-Driven Smart Energy Management System for Polar Research Stations · Ministry of Earth Sciences (MoES) / NCPOR · Software · Clean & Green Technology

1. Context & framing

The PS text is deliberately thin, so the framing is ours to choose — and the framing decides whether we win.

Our framing: NCPOR's stations (Maitri, Bharati in Antarctica; Himadri in the Arctic) are electrical islands resupplied with diesel **once a year, by ship**. That single fact dominates everything. Fuel is not a cost line — it is the survival margin. So we are not building a green dashboard. We are building a **fuel-survival optimizer**, with a days-of-autonomy countdown as the hero metric.

The technical insight the whole pitch rests on

A diesel genset at 25 % load burns roughly **1.5–2× more fuel per kWh** than at 75 %, and below ~30 % it wet-stacks and damages itself. But you cannot simply switch it off, because its **waste heat** is what heats the buildings and melts snow for water. Electricity and heat are **coupled** — optimise them separately and you get the wrong answer. Co-optimising them is the core of our solution, and the thing competing teams will miss.

Locked decisions

- **Station modelled:** Bharati — year-round crew, modern, best documented.
- **Centrepiece:** fuel-survival optimizer. **Engine:** rule-based + MILP hybrid.
- **Hardware-in-the-loop:** optional stretch goal only, after software is locked.
- **Stage:** internal hackathon + idea PPT first, built to extend to the December Grand Finale without rework. SPOC has approved; dates pending.
- **Product name:** team to choose — **HIMSHAKTI** (हिम snow + शक्ति power) or **KAVACH** (कवच armour/shield).

2. Why the optimizer, and not the other three options

This is a decision about **what leads the pitch and where the depth goes** — not about what we cut. Forecasting and blizzard-response are both still built. They just aren't the headline.

Why not forecasting as the centrepiece?

A forecast answers "*what will happen*." It never answers "*so what do we do about it*." The first question any judge asks is "and then?" — if the answer is a chart, we've lost. Worse, forecasting is the **commodity** choice: most teams picking this PS will build exactly this, putting us in a comparison on MAPE, a number NCPOR does not care about. We get forecasting anyway as Layer 2. We just don't lead with it.

Why not blizzard / crisis resilience?

Dramatic, but narrow. A blizzard is an edge case lasting a few days a year; NCPOR's actual problem is 365 days of fuel discipline. It is also hard to quantify — "*the crew survived*" is not a number that fits on a slide, and unquantified claims lose to measured ones. In our design this is a **scenario button**, not a product: we still get the drama on stage, we just don't build the company around it.

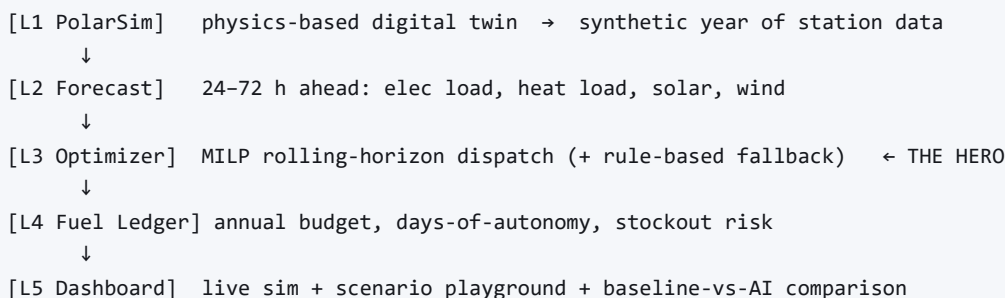
Why not all of them, equally weighted?

This is the classic way strong teams lose. A SIH pitch is a few minutes long — split evenly across four capabilities, each gets about a minute, and nothing is memorable. Judges retain **one number**. Equal weighting also splits build time into four mediocre modules instead of one excellent one with three solid supports. Against domain experts, depth beats breadth every time.

Why the fuel-survival optimizer wins

1. It is the only option producing a **single defensible number tied to the mission** — extra days of autonomy. Everything else produces charts.
2. It is where **EEE knowledge is a moat**: genset efficiency curves, minimum stable load, N-1 reserve, waste-heat coupling. A CS-heavy team cannot easily reach this, and MoES judges will recognise it.
3. It addresses the PS's own third clause — "*fuel optimization*" — which is also NCPOR's real operational pain, given one resupply ship a year.
4. **It subsumes the others.** Forecasting becomes its input; blizzard response becomes a scenario it handles. Choosing it loses us nothing and focuses everything.

3. Architecture — five layers



L1 — PolarSim (digital twin)

Why it exists: there is no public NCPOR energy dataset — nobody at SIH will have real telemetry. So we write a physics-grounded simulator, calibrate it against published Antarctic station energy studies, and state this openly in the PPT as a methodology choice, not an apology. It also lets us demo a blizzard on stage, which real data never would. Generates 1 year at 15-min resolution:

- **Weather** — temperature, windspeed, solar elevation with true polar-night / midnight-sun cycles; blizzards as correlated wind/temperature excursions.
- **Building thermal** — heat loss $Q = U \cdot A \cdot \Delta T$ plus ventilation and infiltration; snow-melt water demand vs crew size.
- **Occupancy** — summer campaign (~50–60) vs winter-over (~20–25). Biggest single load driver.
- **Load library** — every load tagged *critical* / *deferrable* / *sheddable*, with a power profile and, for deferrables, a completion deadline.
- **Assets** — 2–3 gensets with non-linear fuel curves (L/h vs load factor), waste-heat recovery, backup boiler, cold-derated battery, PV with snow soiling, wind turbine with cut-in/cut-out and icing lockout.
- **Event injection** — `blizzard(days)`, `genset_failure(id)`, `resupply_delay(days)`, `crew_surge(n)`.

L2 — Forecast

- Targets: electrical load, thermal load, PV availability, wind availability. Horizon 72 h, hourly.
- LightGBM on lag features + rolling stats + weather + occupancy + calendar. Small, trains in seconds, runs on station hardware with no GPU — say this, it's a credibility point.
- **Always report against a naive baseline** (persistence / seasonal-naive). The lift is the story, not the absolute number.
- Emit **P10/P50/P90** quantiles, not a point forecast — the optimizer needs the uncertainty to size reserve.

L3 — Optimizer (build this before the forecaster)

Rolling-horizon MILP, 48 h lookahead, 1 h steps, re-solved each step. OR-Tools CP-SAT or PuLP/CBC.

- **Decisions:** genset on/off (binary) and loading per unit; battery charge/discharge; deferrable load start times; boiler firing; renewable curtailment.
- **Objective:** minimise litres burned, plus penalties for genset starts (wear), unmet thermal comfort, and unserved critical load (effectively infinite).
- **Constraints that carry the pitch:** electrical balance; **thermal balance including recovered waste heat** (the coupling — our differentiator); genset minimum stable load (anti wet-stacking) and min up/down times; battery SoC with **cold-temperature derating**; deferrable deadlines; **N-1 spinning reserve** (survive the largest genset tripping).
- **Rule-based fallback:** a deterministic priority dispatcher that always returns a feasible answer. If the solver stalls on stage, the demo continues. Not a hack — a life-safety system genuinely needs a deterministic path, so we present it as a design virtue.
- **Baseline for comparison:** "always-on largest genset, no load shifting" — what stations realistically do today. Every saving we claim is measured against this.

L4 — Fuel Ledger · L5 — Dashboard

- Annual fuel burndown against the next resupply date; **days-of-autonomy** as the hero number; Monte Carlo over weather and occupancy → **probability of stockout before resupply**.
- What-if: *"ship delayed 21 days — do we survive?"* Answered with a number, in front of the judges.
- Dashboard hero panel: days-of-autonomy gauge + AI-vs-baseline fuel curves diverging over the year. Then accelerated sim clock, stacked dispatch chart (genset / PV / wind / battery), forecast-vs-actual.
- **Scenario buttons:** Blizzard · Genset Failure · Resupply Delayed · Crew Surge.
- **Money slide, on screen:** litres saved, % reduction, **extra days of autonomy gained**, CO₂ avoided.

4. Stack

Layer	Choice	Why
Core	Python 3.11	simulator, ML and optimizer in one language
Optimizer	OR-Tools CP-SAT (fallback PuLP/CBC)	fast, free, no licence
ML	LightGBM + pandas / numpy	CPU-only, seconds to train
API	FastAPI + WebSocket	streams live sim ticks to the UI
Frontend	React + Vite + Recharts + Tailwind	fast to build, looks finished
Storage	SQLite / Parquet	zero-config, offline-first

Offline-first is a required talking point. Stations have thin, expensive satellite links. Everything runs on-prem on modest hardware — no cloud, no GPU. Say it explicitly; MoES judges will care.

5. Build phases

- Phase 0 — Foundation (½ day).** Repo scaffold, station-profile config schema, shared timestep data model.
- Phase 1 — PolarSim (1½ days).** Weather, thermal, occupancy, load library, assets, event injection. Ship a generated year as Parquet plus plots that look physically plausible. *Gate: the annual profile must be defensible to an EEE professor.*
- Phase 2 — Optimizer + baseline (2 days) ← highest value, do it early.** Rule-based dispatcher first, then MILP, then baseline, then the comparison harness. Initially run with **perfect foresight** (feed it the simulator's true future) so it can be built and validated before the forecaster exists. *Gate: MILP beats baseline on fuel by a clear, reproducible margin.*
- Phase 3 — Forecast (1 day).** Train models, swap them in behind the same interface perfect-foresight used. Measure how much of the saving survives real forecast error — **report that honestly**; it signals rigour, and judges probe for it.
- Phase 4 — Dashboard (2 days).** Hero panel, then dispatch charts, then scenario buttons. Do not start the UI until Phase 2 produces numbers worth displaying.

Phase 5 — Pitch (1 day). PPT in SIH format, a rehearsed 3-minute demo script, and a recorded fallback video in case the laptop or venue Wi-Fi fails.

Stretch (finale only). ESP32 + current sensors + relays driving a lamp/heater as a physical micro-grid, commanded by our controller. Powerful for an EEE team — but only after software is locked.

6. Idea PPT — section map

1. **Problem** — one year of fuel, one ship, no second chance. Lead with the constraint, not the technology.
2. **Solution** — the five layers, one diagram.
3. **Differentiator** — heat/electricity coupling and genset low-load inefficiency. State plainly that electrical-only optimisation is wrong, and why.
4. **Technical approach** — stack, MILP formulation sketch, forecast lift over baseline.
5. **Feasibility** — offline, CPU-only, retrofits onto existing meters; deterministic fallback for life safety.
6. **Impact** — litres saved → days of autonomy → reduced resupply risk → CO₂ and black-carbon reduction in a protected environment (Antarctic Treaty / Madrid Protocol framing).
7. **Research** — published Antarctic station energy studies, NCPOR expedition reports, Maitri II's stated green-station goal. Show the simulator is calibrated, not invented.

7. Verification

- **Physical sanity** — annual energy per capita and fuel burn land inside the range published for comparable Antarctic stations. If the sim says a 25-person station burns 20 L/day, it's wrong.
- **Energy balance** — at every timestep, generation + discharge = load + charge + losses, within tolerance. Assert it in a test: a violated balance invalidates every saving we claim.
- **Optimizer correctness** — on a small hand-checkable instance, MILP output matches a manually computed optimum. Confirm genset minimum-load and N-1 reserve are never violated across a full year.
- **The saving is real** — same year, same weather seed, baseline vs AI; report litres and days gained across **≥10 random seeds** so the headline isn't a lucky draw.
- **Forecast honesty** — MAPE against persistence, and the saving measured under real forecast error, not perfect foresight.
- **Demo resilience** — full run-through with Wi-Fi disabled; kill the solver mid-demo and confirm the rule-based fallback visibly takes over.

8. Open items for the team

- **Product name** — HIMSHAKTI or KAVACH.
- **Internal-hackathon date** from the SPOC, so Phases 4–5 can be scheduled backwards from it.